

תרגיל בית 3: מסלול ממרפסת

שבוע 3 – הגשת קטע

מטלת החובה: זריקה ממרפסת

נרחיב את הקליע הנדגם מן ההרצאה לגובה התחלתי, מהירות, זווית וצעד זמן חיובי שסופקו על ידי המשתמש. נשמור על y כלפי מעלה ונקדם את המצב הישן בשיטת אוילר המפורשת. נשמור היסטוריות מתואמות של y זמן, מיקום ושני רכיבי המהירות, לרבות המצב ההתחלתי והדגימה הראשונה שבה $y \leq 0$. נעבוד בארבעה שלבים. תחילה נשרטט את המסלול החזוי, נציין יחידות וסימנים, ונחשב זמן נחיתה אנליטי להשוואה. לאחר מכן נבנה את הווקטורים ונכתוב את הלולאה, תוך בדיקה ידנית של העדכון הראשון. אחר כך נוודא שלכל הווקטורים אותו אורך, שהזמן גדל בצעד שצוין, שהמהירות האופקית נשארת קבועה ושהמהירות האנכית קטנה ב- $g\Delta t$. לבסוף נחזור על החישוב עם חצי מצעד הזמן ונציג בטבלה את זמני וטווחי הנחיתה הנדגמים לצד הערכים האנליטיים. על הדוח להבחין בין דגימת חציית קרקע לבין פגיעה מדויקת או פגיעה מאונטרפולת. איננו חותכים את הגובה השלילי האחרון ואיננו מחליפים בשקט את הנחיתה הנומריה בתשובה האנליטית. נפעל לפי מוסכמת הקרקע ההתחלתית של המטלה, לרבות מקרה שיגור כלפי מעלה, במקום להניח שהתנאי `while y[end] > 0` מטפל בכל תנאי התחלה. דף שיעורי הבית מפרט את המקטע הנומרי, את הבדיקות הגלויות ואת תוצרי המסלול ועידון צעד הזמן הנבדקים בנפרד. הגדרת פונקציות נשארת חומר של שבוע 5.

תיבת פיסיקה: תנועת קליע נדגמת

זריקה ממרפסת היא קליע נקודתי תחת כבידה אחידה בקירוב וללא התנגדות אוויר. נבחר x אופקי ו- y כלפי מעלה, ולכן $a_x = 0$ ו- $a_y = -g$ עם $g > 0$ ב- m/s^2 ; המצב הוא (t, x, y, v_x, v_y) כאשר הזמן בשניות, המקומות במטרים והמהירויות ב- m/s . שיטת אוילר המפורשת מקדמת את המצב לפי $x_{n+1} = x_n + v_{x,n}\Delta t$, $y_{n+1} = y_n + v_{y,n}\Delta t$ ואז $v_{x,n+1} = v_{x,n}$ ו- $v_{y,n+1} = v_{y,n} - g\Delta t$. הנקודה האחרונה שנשמרת היא הגובה הנדגם הראשון עם $y \leq 0$, ולא חציית הקרקע המדויקת. למקרה ההרצאה $v_0 = 10 \text{ m/s}$, $\theta = 45^\circ$, $y_0 = 1 \text{ m}$, $g = 9.81 \text{ m/s}^2$ יש זמן נחיתה אנליטי של כ- 1.57135 s וטווח של כ- 11.111 m ; חלוקת Δt בחצי אמורה לקרב את הערכים הנדגמים אליהם. עידון הצעד בודק את הדיסקרטיזציה בלבד ואינו יכול להוסיף את הגרר, הניתור או פגיעת הקיר שהושמטו.

המשימה

נדמה קליע המשוגר ממרפסת ונשמור את ההיסטוריה הנדגמת שלו. נבחר x אופקי ו- y כלפי מעלה, $a_y = -g$, $a_x = 0$, ונשתמש בעדכוני אוילר קדמיים: מעדכנים מיקום לפי המהירות הישנה, ואז מעדכנים מהירות. נתחיל במצב ההתחלתי ונוסיף דגימות עד, כולל, הדגימה הראשונה שבה `y[end] <= 0`. נשמור וקטורים מתואמים t, x, y, v_x ו- v_y ; נציב `landing_time = t[end]`. התוצאה היא נחיתה נדגמת ולא פגיעה מדויקת; המודל של כבידה אחידה ללא גרר משמיט התנגשות והתנגדות אוויר. נאמת שהשמות המסופקים הם ממשיים סופיים שאינם בוליאניים, ואז נדרוש `height_m >= 0` ו-`dt_s > 0`, `speed_m_s >= 0` ו-`gravity_m_s2 > 0` (Julia מתייחסת ל-`Bool` כ-`Real`, ולכן נדחה אותו במפורש). קלט לא תקין מעלה `ArgumentError`. עבור שיגור מגובה אפס, המצב ההתחלתי הוא גם הדגימה הראשונה הנדרשת שבה `y <= 0`.

הגשה

נגיש את המקטע הזה בלבד; פונקציות מתחילות בשבוע 5.

```
# validate inputs; initialize and fill the five vectors
# assign landing_time
```

בדיקה

לפני שנסיים את המשימה, כדאי לבדוק מה תוצאה נכונה צריכה לקיים. לחמשת הווקטורים צריך להיות אותו אורך, הזמן צריך להתחיל ב- $t[1] == 0.0$ ולהתקדם ב- dt_s , והגובה האחרון שנשמר צריך להיות הלא-חיובי הראשון. נריץ את המקטע פעם עם dt_s ופעם עם $dt_s / 2$, ואז נשווה את זמני וטווחי הנחיתה הנדגמים; כך רואים בכמה זיה התוצאה כשצעד הזמן קטן. עידון צעד הזמן בודק את הרגישות הזו בלבד – הוא אינו מחזיר את פיסיקת הפגיעה או התנגדות האוויר שהמודל שלנו משמיט.

פונקציה עוטפת ובדיקות מתקדמות, רשות

הפונקציה העוטפת המסופקת היא `balcony_trajectory(height_m, speed_m_s, angle_deg, dt_s; gravity_m_s2=9.81)` והיא מחזירה `(t, x, y, vx, vy, landing_time)`. אם נרצה לראות כיצד המערכת בודקת את העבודה שלנו, הקריאה האופציונלית מכניסה את `Test` `using`: `@test` מצהירה על טענה שחייבת להתקיים, ו-`@test_throws ArgumentError` מצהיר שקלט לא תקין מתועד חייב להידחות. לראות את הטענות האלה, ולהריץ אותן בעצמנו, זה מה שהופך את הבדיקה לניתנת לשחזור ולא לעניין של אמון. בדיקות הסגל משתמשות בחוזה המפורסם הזה בלבד ואינן מוסיפות דרישות משל עצמן. גוף הבדיקות הגלויות המלא הוא:

```
run_question_tests("week03 core public") do
    t, x, y, vx, vy, landing = balcony_trajectory(2.0, 10.0, 45.0, 0.1)
    @test length(t) == length(x) == length(y) == length(vx) == length(vy)
)
    @test t[1] == 0.0 && y[1] == 2.0 && landing == t[end]
    @test y[end] <= 0 && y[end - 1] > 0
    @test all(isapprox.(diff(t), 0.1))
    @test balcony_trajectory(0.0, 5.0, 30.0, 0.1)[1:3] == ([0.0], [0.0],
[0.0])
    @test_throws ArgumentError balcony_trajectory(-1.0, 1.0, 0.0, 0.1)
    @test_throws ArgumentError balcony_trajectory(1.0, 1.0, 0.0, 0.0)
    @test_throws ArgumentError balcony_trajectory(true, 1.0, 0.0, 0.1)
end
```